Multi-Agent Coordination Implementation Guide

How Sub-Agents Work Together as a Team

with Better Outcomes Using TodoBoard Coordination

Deep Research Report

Research Date: 2026-03-05

Sources dated from 2024-01 to 2026-03

Prepared by: II Deep Research Agent

Based on: pi-mono/coding-agent harness architecture

Reference: Hive Mind Multi-Agent Coordination Theory

Table of Contents

1 Executive Summary ... 4
2 Introduction ... 5
2.1 Background ... 5
2.2 Scope ... 5
2.3 Key Research Questions ... 5
3 Multi-Agent Coordination Architecture Patterns ... 7
3.1 The Supervisor-Subagent Model ... 7
3.2 Hierarchical Goal Decomposition ... 7
3.3 Agent Role Specialization ... 8
3.4 Communication Topology Patterns ... 8
4 TodoBoard-Based Task Coordination ... 10
4.1 The Role of Shared Task State ... 10
4.2 The tick-md Model: Tasks as Markdown ... 10
4.3 Implementing TodoBoard for Sub-Agents ... 11
4.4 TodoBoard Benefits for Multi-Agent Coordination ... 12
5 Shared Memory and State Management ... 14
5.1 The Need for Collective Memory ... 14
5.2 Implementing Shared Memory with Access Control ... 14
5.3 Event Sourcing for Multi-Agent State ... 15
5.4 Context Compaction Strategies ... 16
6 Communication Protocols and Delegation ... 17
6.1 Contract Net Protocol for Task Delegation ... 17
6.2 Bounded Context Delegation ... 17
6.3 Steering and Follow-Up Messages ... 18
6.4 Consensus Mechanisms for Critical Decisions ... 18
7 Production Deployment Strategies ... 20
7.1 Token and Time Boundaries ... 20
7.2 Resource Contention Detection ... 20
7.3 Runtime Guardrails ... 20
7.4 Workflow Checkpointing and Rollback ... 21
7.5 Continuous Evaluation and Learning ... 21
8 Implementation Recommendations for pi-mono Harness ... 23
8.1 Extending the Subagent Architecture ... 23
8.2 Git Coordination Enhancements ... 25
8.3 AGENTS.md Enhancements for Multi-Agent Coordination ... 25
8.4 Performance Monitoring Dashboard ... 26
9 Analysis and Discussion ... 27
9.1 Synthesis of Coordination Patterns ... 27
9.2 Trade-offs and Considerations ... 27
9.3 Limitations of This Research ... 27
10 Conclusions ... 28
10.1 Key Findings Summary ... 28
10.2 Recommendations ... 28
10.3 Future Research Directions ... 28
11 References ... 30
12 Appendix A: TodoBoard Schema Reference ... 31

13 Appendix B: Coordination Event Types 32
14 Appendix C: Multi-Agent Coordination Checklist 33

1 Executive Summary

This report provides a comprehensive implementation guide for building multi-agent systems where sub-agents collaborate effectively as a coordinated team. Drawing from extensive research on coordination theory, production-grade frameworks, and real-world implementations, this guide addresses the critical challenge of transforming independent AI agents into cohesive, high-performing teams that deliver superior outcomes compared to single-agent approaches.

The core finding is that effective multi-agent coordination requires three fundamental pillars: (1) **deterministic task allocation** through shared task management systems like TodoBoards, (2) **hierarchical goal decomposition** with clear role specialization, and (3) **shared memory with access control** enabling agents to build on each other's work without conflicts. The research synthesizes insights from the pi-mono/coding-agent harness architecture (GitHub Repository), the tick-md Markdown-based coordination system (Purple Horizons), and industry frameworks including CrewAI, AutoGen, and LangGraph to provide actionable implementation guidance.

Key recommendations include: implementing a TodoBoard-style task management layer that provides single-source-of-truth coordination visible to all agents; adopting the supervisor-subagent architectural pattern with explicit delegation mechanisms; establishing token and time boundaries to prevent coordination deadlocks; deploying consensus mechanisms for critical decisions; and integrating persistent memory systems that enable agents to learn from collective experience. The guide provides specific code patterns, coordination protocols, and production deployment strategies that teams can immediately apply to transform their multi-agent systems from chaotic independent actors into synchronized, high-output teams.

2 Introduction

2.1 Background

The evolution of Large Language Model (LLM) agents from single-purpose assistants to complex multi-agent systems represents one of the most significant architectural shifts in AI application development. While individual LLM agents have demonstrated remarkable capabilities in reasoning, code generation, and task execution, real-world problems increasingly demand the coordinated efforts of multiple specialized agents working in concert (SuperAnnotate, 2025).

The fundamental challenge is coordination. When multiple autonomous agents operate within the same environment—whether that environment is a codebase, a research project, or a complex business workflow—the potential for conflicts, duplicated effort, and miscommunication grows exponentially. Research indicates that multi-agent systems can consume 1.5x to 7x more tokens than theoretically necessary due to coordination overhead, with token duplication rates reaching as high as 86% in some frameworks (Galileo AI, 2025).

The pi-mono/coding-agent harness represents a practical implementation of multi-agent coordination principles, featuring subagent extensions, manager components, and explicit Git-based coordination rules designed for parallel agent work (AGENTS.md). This architecture, combined with insights from the Hive Mind Multi-Agent Coordination Theory documentation, provides a foundation for understanding how TodoBoard-style task management can transform agent coordination from a bottleneck into a competitive advantage.

2.2 Scope

This research report covers the following areas:

- **Coordination Architecture Patterns**: Analysis of supervisor-subagent, peer-to-peer, and hybrid coordination models with specific implementation guidance for the pi-mono harness environment
- **TodoBoard Task Management**: Deep dive into how shared task management systems enable deterministic task allocation, prevent conflicts, and provide audit trails for multi-agent workflows
- **Communication Protocols**: Examination of Contract Net Protocol, emergent communication, and event-driven architectures for inter-agent messaging
- **State Management**: Event sourcing, CQRS patterns, and shared memory architectures that enable agents to maintain consistent state
- **Memory Integration**: Hierarchical memory systems, context compaction strategies, and persistent memory layers that allow agents to learn collectively
- **Production Deployment**: Runtime guardrails, consensus mechanisms, and fail-safe rollback strategies for production multi-agent systems

This report does **not** cover: single-agent optimization techniques, LLM fine-tuning methodologies, or infrastructure-level concerns such as GPU scaling and model serving.

2.3 Key Research Questions

This report addresses the following questions:

1. **How can sub-agents be structured to work together as a coordinated team rather than independent actors?** This includes examining role specialization, communication patterns, and delegation mechanisms.

2. **What role does TodoBoard-style task management play in multi-agent coordination?** This explores how shared, visible task state prevents conflicts and enables deterministic task allocation.

3. **What coordination patterns from distributed systems theory apply to multi-agent LLM systems?** This draws from Byzantine Fault Tolerance, consensus mechanisms, and collective memory systems.
4. **What are the production-grade strategies for preventing coordination failures?** This covers runtime guardrails, resource contention detection, and fail-safe mechanisms.
5. **How can the pi-mono/coding-agent harness be extended with effective subagent coordination?** This provides specific implementation recommendations for the target architecture.

3 Multi-Agent Coordination Architecture Patterns

3.1 The Supervisor-Subagent Model

The most prevalent and effective architecture for multi-agent LLM systems is the supervisor-subagent (or orchestrator-worker) pattern. In this model, a central supervisor agent receives high-level tasks, decomposes them into subtasks, delegates work to specialized subagents, collects results, and synthesizes final outputs (VoltAgent, 2026).

The supervisor-subagent model addresses several fundamental coordination challenges. First, it provides **centralized task allocation**, eliminating the "ping-ponging" behavior where multiple agents compete for or ignore the same task due to ownership ambiguity (Galileo AI). Second, it creates **natural communication boundaries**: subagents communicate primarily with their supervisor rather than directly with each other, reducing the complexity of n-to-n communication patterns to a more manageable hub-and-spoke topology. Third, it enables **hierarchical decomposition**: complex goals can be recursively broken down through multiple layers of supervisors, each managing a team of specialists.

The pi-mono harness implements this pattern through its subagent extension architecture. According to the commit history, the system includes "subagent extension in addons-extensions/ for self-registration," with the "subagent module exported from main index.ts" (packages/coding-agent). The architecture supports automatic supervisor configuration that "revises the supervisor's system message to include instructions on treating its subagents" and provides a `delegate_task` tool for explicit delegation.

A typical supervisor workflow follows this sequence: (1) the supervisor receives a user request; (2) it analyzes the request to determine which subagents should handle it; (3) it uses the delegation mechanism to assign subtasks with appropriate context; (4) subagents execute their assignments and return structured results; (5) the supervisor combines results and produces the final output. This pattern is analogous to the Contract Net Protocol from distributed systems theory, where a manager announces tasks and contractors bid on them, but with the supervisor making unilateral assignment decisions rather than accepting bids (Contract Net Protocol).

3.2 Hierarchical Goal Decomposition

Effective multi-agent systems replace "chaotic peer-to-peer attempts to solve entire problems with a parent-child chain of responsibility" (Galileo AI). This hierarchical decomposition follows the DEPART framework pattern: **D*ivide (break goals into subgoals),** E*valuate (assess feasibility and dependencies), **P*lan (sequence execution),** A*ct (execute subtasks), **R*eflect (assess results), and** T*rack (maintain progress state).

The Hive Mind documentation describes Hierarchical Task Network (HTN) Planning as the formal foundation for this approach. In HTN planning, compound tasks are decomposed into primitive tasks through methods—rules that specify how to break down abstract tasks based on preconditions. The key insight is that "HTN planning searches through decomposition space rather than action space," using domain-specific knowledge encoded in methods to guide the search efficiently.

For practical implementation, hierarchical decomposition requires:

Layer 1 - Strategic Planning: A top-level agent determines "what matters most" and establishes priority ordering. This agent receives high-level goals and decomposes them into 3-5 major subgoals. In the travel planning example from SuperAnnotate, this might involve decomposing "Plan a trip to Tokyo" into subgoals for flights, hotels, transportation, and activities (SuperAnnotate).

Layer 2 - Tactical Planning: Mid-level agents handle task sequencing and resource allocation. They receive subgoals from Layer 1 and create detailed task plans with dependencies. These agents may further delegate to specialized executors.

Layer 3 - Execution: Worker agents perform concrete actions—code generation, API calls, document creation, or inference. These agents have narrow, well-defined capabilities and return structured results to their supervisors.

This layered approach provides **localized containment**: when a Layer 3 agent fails, the failure can be handled at Layer 2 without propagating to the entire system. It also enables **graceful degradation**: under resource constraints, the system can favor simpler agent configurations or more aggressive context compaction without losing high-level coordination.

3.3 Agent Role Specialization

The CrewAI framework emphasizes that "roles support clean, scalable task decomposition and agent collaboration" (DigitalOcean). Each agent in a well-designed multi-agent system should have a clearly defined role, goal, and set of capabilities. This specialization provides several benefits.

Reduced cognitive load: By limiting each agent's scope, we reduce the complexity of prompts and the likelihood of hallucinations. An agent focused solely on "analyzing code for security vulnerabilities" can maintain deeper expertise than a general-purpose agent attempting to handle all coding tasks.

Optimized model selection: Different roles may warrant different underlying models. A supervisor agent requiring strong reasoning might use a more capable (and expensive) model, while high-frequency simple tasks can use faster, cheaper models (VoltAgent).

Clear accountability: When each agent has a defined role, tracing errors and attributing outputs becomes straightforward. This is critical for debugging multi-agent systems where "a single agent's mistake can affect the whole process and be hard to trace."

The pi-mono AGENTS.md file demonstrates role specialization through its Git coordination rules: "ONLY commit files YOU changed in THIS session" and "Track which files you created/modified/deleted during the session" (AGENTS.md). These rules establish clear boundaries that prevent agents from interfering with each other's work.

3.4 Communication Topology Patterns

The Hive Mind documentation outlines four primary coordination topologies, each with distinct characteristics for multi-agent systems:

Star Topology: A central node connects to all other agents. This provides efficient centralized control and simple implementation but creates a single point of failure. Best suited for small swarms with tight coordination requirements.

Mesh Topology: Each agent connects to multiple neighbors, creating robust and redundant communication paths. This topology supports decentralized decision-making and scales to large swarms but introduces coordination complexity.

Ring Topology: Agents form a closed loop, each connected to two neighbors. This provides simple, predictable flow suitable for cyclic or sequential tasks but offers limited communication efficiency.

Tree/Hierarchical Topology: Parent-child relationships create clear authority flow. This topology is scalable and matches the supervisor-subagent model but can be affected by intermediate node failures.

For most multi-agent LLM applications, the **hierarchical topology** combined with limited **mesh connectivity** between peer specialists provides the best balance. The supervisor-subagent structure

handles coordination and task delegation, while allowing specialized agents working on related subtasks to share intermediate results directly without routing through supervisors.

4 TodoBoard-Based Task Coordination

4.1 The Role of Shared Task State

When multiple AI agents work on the same project, coordination becomes the bottleneck (Purple Horizons). The fundamental insight behind TodoBoard-style coordination is that **shared, visible task state** transforms chaotic parallel execution into deterministic, conflict-free collaboration.

A TodoBoard (or task management system) serves as the single source of truth for what needs to be done, who is doing it, and what has been completed. This addresses several critical coordination challenges:

Task ownership ambiguity: Without explicit ownership, multiple agents may independently decide to work on the same task, wasting resources and creating merge conflicts. A TodoBoard with claiming mechanisms ensures each task has exactly one owner at any time.

Progress visibility: Agents need to know what others are doing to avoid dependencies on incomplete work. A shared task board provides real-time visibility into the state of all tasks.

Audit trail and debugging: When something goes wrong, understanding the sequence of events is critical. TodoBoards that track task state changes provide an audit trail for debugging coordination failures.

Asynchronous coordination: Agents may operate at different times, in different sessions, or with different tools. A persistent TodoBoard enables coordination "across time, sessions, and different tools —like a distributed team where nobody is online at the same time" (Purple Horizons).

The pi-mono harness philosophy explicitly states "No built-in to-dos" but recommends "use TODO.md or build with extensions" (packages/coding-agent README). This extensibility-first approach allows teams to implement TodoBoard coordination tailored to their specific needs.

4.2 The tick-md Model: Tasks as Markdown

The tick-md project provides an elegant implementation of TodoBoard coordination for AI agents using a single Markdown file (TICK.md) as the task database (Purple Horizons). This approach is "AI-native" because "agents can read and write Markdown natively; no custom parsers or APIs required."

The tick-md coordination model includes:

Task lifecycle management: Tasks progress through states including creation, claiming, in-progress, and completion. Each transition is explicit and recorded.

Claiming mechanism: When an agent claims a task, the system locks the file to prevent concurrent edit conflicts. This file locking prevents "race conditions when two agents tried to claim tasks simultaneously."

Dependency tracking: Tasks can block other tasks. When a blocker task completes, dependent tasks automatically unblock.

Git-based audit trail: Every action becomes a Git commit with timestamp and author. This provides complete transparency and enables rollback if coordination breaks down.

The CLI workflow demonstrates the simplicity of the model:

```
tick init                                   # Initialize in project
tick add "Build authentication system" --priority high --tags backend
tick claim TASK-001 @agent-name             # Agent claims task
tick done TASK-001 @agent-name              # Agent marks complete
```

This model maps directly to the deterministic task allocation strategy recommended by Galileo AI: "Unique task IDs, logging the chosen agent, and rejecting reassignment unless explicitly released neutralize under-specification and role-ambiguity" (Galileo AI).

4.3 Implementing TodoBoard for Sub-Agents

For the pi-mono harness environment, implementing TodoBoard coordination requires integrating task management into the extension system. Based on the research, an effective implementation would include:

1. Task Schema Definition

Define a consistent schema for tasks that all agents can read and write:

```
interface Task {
  id: string;                    // Unique identifier (TASK-001)
  content: string;               // Task description
  status: 'pending' | 'in_progress' | 'completed' | 'blocked';
  priority: 'low' | 'medium' | 'high';
  assignee?: string;             // Agent name or ID
  blockedBy?: string[];          // Task IDs this depends on
  createdAt: string;             // ISO timestamp
  updatedAt: string;             // ISO timestamp
  completedAt?: string;          // ISO timestamp
  notes?: string[];              // Progress notes
}
```

2. Claiming Protocol

Implement atomic claiming to prevent race conditions:

```
async function claimTask(taskId: string, agentId: string): Promise<boolean> {
  // Acquire file lock
  const lock = await acquireLock('TODO.md');
  try {
    const tasks = await readTasks();
    const task = tasks.find(t => t.id === taskId);

    if (task.status !== 'pending') {
      return false;  // Already claimed
    }

    task.status = 'in_progress';
    task.assignee = agentId;
    task.updatedAt = new Date().toISOString();

    await writeTasks(tasks);
    await gitCommit(`Claimed ${taskId} by ${agentId}`);
    return true;
  } finally {
    await releaseLock(lock);
  }
}
```

3. Supervisor Integration

The supervisor agent uses the TodoBoard to coordinate subagents:

```
// Supervisor workflow
async function supervisorWorkflow(goal: string) {
```

```
  // Decompose goal into tasks
  const subtasks = await decomposeGoal(goal);

  // Add tasks to TodoBoard
  for (const subtask of subtasks) {
    await addTask(subtask);
  }

  // Delegate to subagents
  for (const subagent of availableSubagents) {
    const pendingTasks = await getTasksForCapability(subagent.capability);
    if (pendingTasks.length > 0) {
      await delegateTask(subagent, pendingTasks[0]);
    }
  }

  // Monitor and collect results
  await waitForCompletion();
  return synthesizeResults();
}
```

4. Progress Tracking

Agents add progress notes while working, enabling other agents (and humans) to understand current state:

```
async function updateProgress(taskId: string, note: string) {
  const tasks = await readTasks();
  const task = tasks.find(t => t.id === taskId);

  task.notes = task.notes || [];
  task.notes.push(`[${new Date().toISOString()}] ${note}`);
  task.updatedAt = new Date().toISOString();

  await writeTasks(tasks);
}
```

4.4 TodoBoard Benefits for Multi-Agent Coordination

Research and practical implementations demonstrate several concrete benefits of TodoBoard-based coordination:

Elimination of duplicate work: The tick-md team reports that after implementing file locking and claiming, they eliminated race conditions where "two agents tried to claim tasks simultaneously" (Purple Horizons).

Predictable assignment: Using deterministic schemes such as "round-robin queues, capability-rank sorting, or elected leaders lets every agent infer the same assignment without negotiation" (Galileo AI). The TodoBoard provides the shared state needed for these schemes.

Transparency and debugging: When all task state is visible and version-controlled, debugging coordination failures becomes straightforward. "Anyone can open TICK.md to see current state—no hidden state outside the file" (Purple Horizons).

Offline and asynchronous operation: TodoBoards stored as files work offline and with any Git provider. Agents can coordinate "across time, sessions, and different tools" without requiring real-time connectivity.

Human-AI collaboration: Humans can easily inspect, modify, and add tasks to the TodoBoard using standard tools. This integration supports the pi-mono philosophy of human+AI collaboration.

5 Shared Memory and State Management

5.1 The Need for Collective Memory

Multi-agent systems require shared memory to avoid information silos "where agents miss context discovered earlier" (Galileo AI). Without shared memory, Agent A might spend time researching a topic that Agent B already investigated, or worse, multiple agents might make decisions based on contradictory information.

The Hive Mind documentation describes this through the lens of collective memory systems like dMARS (distributed Multi-Agent Reasoning System), where agents share beliefs through a common knowledge repository. Each agent maintains its own Beliefs, Desires, and Intentions (BDI model), but beliefs are synchronized through the shared repository to ensure consistency.

For LLM-based agents, shared memory takes several forms:

Session memory: Short-term context shared within a conversation or workflow session. This includes intermediate results, current task state, and recent decisions.

Persistent memory: Long-term storage surviving across sessions, including user preferences, learned patterns, and historical summaries.

Workspace/scratchpad: A computational workspace for multi-step analysis, referenced by key rather than embedding all content in active context (Olamendy, Medium).

5.2 Implementing Shared Memory with Access Control

The recommended approach involves vector databases as shared memory with "strict access control lists (ACLs) and role-based namespaces" (Galileo AI). Key implementation principles include:

Namespace isolation: Organize state as hierarchical keys (e.g., `user.preferences.language`, `session.workflow.current_step`, `agent.specialist_A.tool_results`). Give agents selective access to namespaces to enforce least privilege.

Read-only vs. modifiable access: Share read-only context for coordination without granting modification permissions. Use shared memory spaces when collaborative problem-solving requires write access, while maintaining private working memory for each agent.

Timestamps and TTL: Add timestamps to embeddings and enforce time-to-live for stale facts. This prevents agents from acting on outdated information.

Provenance tracking: When an agent writes, attach its role and task ID for auditing and tracing. This enables debugging when conflicts arise.

The practical implementation pattern from the best practices research:

```python
# State management with namespaces
class SharedMemory:
    def __init__(self):
        self.store = VectorDatabase()
        self.acl = AccessControlList()

    def write(self, namespace: str, key: str, value: any, agent_id: str):
        if not self.acl.can_write(agent_id, namespace):
            raise PermissionError(f"Agent {agent_id} cannot write to {namespace}")

        entry = {
            'value': value,
            'agent_id': agent_id,
```

```
            'timestamp': datetime.now().isoformat(),
            'ttl': self.get_ttl(namespace)
        }
        self.store.upsert(f"{namespace}.{key}", entry)

    def read(self, namespace: str, key: str, agent_id: str):
        if not self.acl.can_read(agent_id, namespace):
            raise PermissionError(f"Agent {agent_id} cannot read from {namespace}")

        entry = self.store.get(f"{namespace}.{key}")
        if self.is_expired(entry):
            return None
        return entry['value']
```

5.3 Event Sourcing for Multi-Agent State

The Hive Mind documentation highlights Event Sourcing as a powerful pattern for multi-agent state management. Instead of storing only current state, the system stores all state-changing events. Current state can be reconstructed by replaying events.

Benefits for multi-agent systems include:

Complete audit trail: Every state change is recorded with timestamp and source agent. This provides perfect debugging capability when coordination fails.

Temporal queries: The system can answer questions like "What was the state when Agent X made decision Y?" by replaying events up to that point.

Conflict resolution: When multiple agents attempt conflicting updates, the event log provides the information needed to resolve conflicts and replay with corrections.

Checkpoint and rollback: Strategic checkpoints enable "fail-safe rollbacks" where the system can "restore known-good states when failures occur" (Galileo AI).

The event-driven architecture pattern integrates naturally with multi-agent coordination:

```
// Event types for multi-agent coordination
type AgentEvent =
  | { type: 'TASK_CLAIMED'; taskId: string; agentId: string; timestamp: string }
  | { type: 'TASK_COMPLETED'; taskId: string; agentId: string; result: any }
  | { type: 'OBSERVATION_RECORDED'; agentId: string; observation: any }
  | { type: 'DECISION_MADE'; agentId: string; decision: any; reasoning: string };

// Event store
class EventStore {
  private events: AgentEvent[] = [];

  append(event: AgentEvent) {
    this.events.push(event);
    this.notifySubscribers(event);
  }

  replay(upTo?: string): State {
    let state = initialState();
    for (const event of this.events) {
      state = applyEvent(state, event);
      if (event.timestamp === upTo) break;
    }
    return state;
```

```
  }

  getEventsForAgent(agentId: string): AgentEvent[] {
    return this.events.filter(e => e.agentId === agentId);
  }
}
```

5.4 Context Compaction Strategies

As multi-agent workflows progress, context accumulates and can exceed model context windows. Research identifies several compaction strategies (Olamendy):

Hierarchical summarization: Recent turns remain verbatim for immediate reference. Older turns are summarized into issue-resolution pairs. Ancient conversations are compressed into preference/history summaries.

Multi-level compaction: Operate compaction at conversation level (summarize completed dialogue), workflow level (compact execution traces), and tool level (summarize verbose tool outputs).

Salience marking: Information referenced multiple times is marked as salient and protected from aggressive compaction.

Task-critical preservation: Context identified as task-critical via explicit tagging or learned patterns receives higher retention priority.

The pi-mono harness supports compaction through the `/compact` command and automatic compaction settings. The system "triggers on context overflow or when approaching limit" and is "configurable via /settings or settings.json" (packages/coding-agent).

6 Communication Protocols and Delegation

6.1 Contract Net Protocol for Task Delegation

The Contract Net Protocol (CNP) provides a formal framework for task allocation in multi-agent systems. As described in the Hive Mind documentation, CNP involves a manager agent announcing tasks and contractor agents bidding based on their capabilities. While pure CNP involves competitive bidding, the underlying message flow—announcement, evaluation, assignment, and confirmation—applies directly to supervisor-subagent coordination.

The CNP message sequence for multi-agent LLM systems:

1. Task Announcement: The supervisor broadcasts task requirements to potential subagents. This includes task description, required capabilities, deadline, and evaluation criteria.

2. Capability Assessment: Subagents evaluate their fit for the task based on their defined roles, current workload, and available context.

3. Assignment: The supervisor selects the best-suited subagent based on capability match, availability, and historical performance.

4. Confirmation: The assigned subagent confirms acceptance and provides estimated completion time.

5. Execution and Reporting: The subagent executes the task and returns structured results.

For the pi-mono harness, this protocol is implemented through the `delegate_task` tool that "allows [the supervisor] to delegate tasks to subagents" (VoltAgent). The supervisor's system message is automatically revised to include "instructions on treating its subagents."

6.2 Bounded Context Delegation

A critical best practice for multi-agent communication is **bounded context delegation**: when delegating, the supervisor explicitly defines what context the subagent receives rather than passing the entire conversation history (Olamendy).

Benefits of bounded context delegation include:

Token efficiency: Subagents receive only relevant context, reducing token consumption and latency.

Focus: Subagents are not distracted by irrelevant conversation history or other agents' work.

Privacy: Sensitive information from one subagent's work is not automatically exposed to others.

Encapsulation: The subagent's internal reasoning remains encapsulated; only structured results propagate upward.

Implementation pattern:

```
interface DelegationContext {
  taskDescription: string;
  relevantFacts: string[];
  constraints: string[];
  expectedOutputSchema: OutputSchema;
}

async function delegateWithBoundedContext(
  subagent: Agent,
  task: Task,
  fullContext: ConversationContext
): Promise<TaskResult> {
```

```
  // Extract only relevant context
  const boundedContext: DelegationContext = {
    taskDescription: task.content,
    relevantFacts: extractRelevantFacts(fullContext, task),
    constraints: task.constraints,
    expectedOutputSchema: task.outputSchema
  };

  // Delegate with bounded context
  const result = await subagent.execute(boundedContext);

  // Subagent returns structured result, not full reasoning trace
  return {
    taskId: task.id,
    output: result.output,
    confidence: result.confidence,
    status: result.status
  };
}
```

6.3 Steering and Follow-Up Messages

The pi-mono harness implements a sophisticated message queue system for coordinating agent communication (packages/coding-agent):

Steering messages (Enter key): Delivered after current tool execution, interrupting remaining tools. These allow humans or supervisors to redirect an agent mid-execution.

Follow-up messages (Alt+Enter): Delivered only after the agent finishes all current work. These queue additional tasks without interrupting current execution.

This distinction is crucial for multi-agent coordination:

- Steering messages enable **reactive coordination**: when a supervisor detects that a subagent is heading in the wrong direction, it can intervene immediately.
- Follow-up messages enable **pipelining**: tasks can be queued for execution without waiting for the current task to complete.

The `steeringMode` and `followUpMode` settings (`"one-at-a-time"` or `"all"`) provide fine-grained control over how queued messages are processed.

6.4 Consensus Mechanisms for Critical Decisions

For high-stakes decisions, multi-agent systems benefit from consensus mechanisms that require agreement among multiple agents before proceeding (Galileo AI). The Hive Mind documentation covers Byzantine Fault Tolerance (BFT) as the theoretical foundation, with the key insight that a system with n nodes can tolerate up to f faulty nodes if $n \geq 3f + 1$.

Practical consensus patterns for LLM agents include:

Majority voting: Multiple agents independently analyze a problem, and the majority opinion is adopted. This reduces the impact of individual agent hallucinations.

Weighted confidence voting: Agents provide confidence scores with their outputs, and votes are weighted by confidence. This allows agents with more relevant expertise to have greater influence.

Aggregation agents: A lightweight aggregation agent collects candidate outputs and "records votes and computes agreement scores in real time" (Galileo AI).

Hierarchical consensus: For large systems, consensus is computed within subgroups, and subgroup results are aggregated at higher levels.

Implementation example:

```
interface AgentVote {
  agentId: string;
  decision: string;
  confidence: number;
  reasoning: string;
}

async function achieveConsensus(
  agents: Agent[],
  question: string,
  threshold: number = 0.7
): Promise<ConsensusResult> {
  // Collect votes from all agents
  const votes: AgentVote[] = await Promise.all(
    agents.map(agent => agent.vote(question))
  );

  // Group by decision
  const decisionGroups = groupBy(votes, v => v.decision);

  // Calculate weighted agreement for each decision
  const scores = Object.entries(decisionGroups).map(([decision, votes]) => ({
    decision,
    weightedScore: votes.reduce((sum, v) => sum + v.confidence, 0) /
                   votes.reduce((sum, v) => sum + 1, 0),
    voteCount: votes.length
  }));

  // Check if any decision meets threshold
  const winner = scores.find(s => s.weightedScore >= threshold);

  if (winner) {
    return { decision: winner.decision, confidence: winner.weightedScore };
  } else {
    return { decision: 'NO_CONSENSUS', confidence: 0 };
  }
}
```

7 Production Deployment Strategies

7.1 Token and Time Boundaries

Multi-agent systems can become trapped in "expensive, endless negotiations or prompt-variation debates, burning compute and masking lack of progress" (Galileo AI). Research shows alarming token duplication rates: MetaGPT 72%, CAMEL 86%, and AgentVerse 53%. Multi-agent systems consume 1.5x to 7x more tokens than theoretically necessary.

To prevent coordination deadlocks and runaway costs, implement explicit boundaries:

Step-count caps: Limit the total number of conversation turns per task. When the cap is reached, the agent must conclude or yield.

Elapsed-time ceilings: Ensure tasks finish within defined time limits. Long-running tasks may indicate coordination problems.

Idle-time guards: Eliminate agents that stop responding. This prevents zombied agents from blocking coordination.

Token budgets: Set per-task and per-agent token limits. When budgets are approached, trigger compaction or graceful termination.

The pi-mono harness supports these boundaries through configurable settings and automatic compaction triggers. Extensions can implement custom budget management logic.

7.2 Resource Contention Detection

Resource contention—API rate limits, database locks, GPU starvation—can cause cascading failures across agent workflows (Galileo AI). Production systems must detect and mitigate contention proactively.

Exponential backoff: When encountering rate limits (429 errors) or locks, implement exponential backoff: wait and double the delay on each subsequent attempt.

Observability: Deploy platforms that "ingest trace data from every agent, cluster errors in real time, and surface contention hot spots."

Resource reservation: For critical resources, implement reservation systems that prevent multiple agents from attempting simultaneous access.

Priority queuing: When contention occurs, serve high-priority tasks first and defer or drop lower-priority requests.

The pi-mono AGENTS.md includes implicit contention management through Git coordination rules: specific file-by-file commits prevent merge conflicts, and the explicit prohibition of `git add -A` or `git add .` prevents one agent from accidentally committing another agent's uncommitted changes (AGENTS.md).

7.3 Runtime Guardrails

Runtime guardrails provide "real-time policy enforcement for tool calls, content filtering, action verification, automatic PII redaction, and logging" (Galileo AI). This is critical because prompt injection is classified as the 1 LLM vulnerability (OWASP LLM01:2025).

Guardrail implementation layers:

Content filtering: Screen agent outputs for harmful, biased, or policy-violating content before execution or delivery.

Action verification: Before executing high-impact actions (file deletion, financial transactions, external API calls), verify the action against defined policies.

PII redaction: Automatically detect and redact personally identifiable information from logs and shared state.

Audit logging: Generate audit records suitable for compliance requirements (e.g., HIPAA, SOC 2, GDPR).

The pi-mono harness supports guardrails through the extension system. Extensions can "implement permission gates" and custom validation logic. The philosophy section notes that while pi has "No permission popups" built in, teams can "run in container or build confirmation flow with extensions."

7.4 Workflow Checkpointing and Rollback

The strategy of "capturing complete workflow snapshots at strategic milestones" enables recovery from coordination failures (Galileo AI). Checkpoints function like Git commits for live agent ecosystems.

Implementation recommendations:

Strategic checkpoint placement: Capture checkpoints before high-impact actions and after major dependency boundaries. Do not checkpoint after every action—this creates storage overhead without proportional benefit.

Immutable storage with hash signatures: Store checkpoint artifacts with cryptographic hashes to detect corruption.

Version control for interactions: Enable teams to "identify problematic execution traces and revert without disrupting parallel operations."

Automatic rollback triggers: Define conditions (error rates, consistency check failures) that trigger automatic rollback to the last known-good checkpoint.

The pi-mono session system provides a foundation for checkpointing. Sessions are stored as JSONL files with tree structures (each entry has `id` and `parentId`), enabling branching and history traversal via the `/tree` command. The `/fork` command creates new session files from a selected point, effectively creating a checkpoint (packages/coding-agent).

7.5 Continuous Evaluation and Learning

Static evaluation systems cannot keep pace with evolving coordination patterns and emerging failure modes. The recommended approach is Continuous Learning via Human Feedback (CLHF): "Feed a handful of fresh edge cases each week to retrain evaluators and redeploy" (Galileo AI).

Implementation approach:

Real-time monitoring pipelines: Capture operational signals including coordination timing, error rates, resource usage, and output quality scores.

Failure pattern recognition: Automatically detect coordination breakdowns through anomaly detection on monitoring data.

Feedback loops: Route detected issues to human reviewers who can label examples for retraining.

Evaluator retraining: Periodically retrain evaluation models with fresh labeled examples, focusing on emerging failure modes.

The Mem0 integration for CrewAI demonstrates this pattern: "Adaptive, self-improving memory that continuously refines stored knowledge, updates facts, and filters out irrelevant details over time" (Mem0).

8 Implementation Recommendations for pi-mono Harness

8.1 Extending the Subagent Architecture

Based on the research findings, the following extensions would enhance the pi-mono subagent coordination capabilities:

1. TodoBoard Extension

Create an extension that provides TodoBoard functionality integrated with the existing session and Git systems:

```
// extensions/todoboard/index.ts
export default function(pi: ExtensionAPI) {
  // Register TodoBoard tool for agents
  pi.registerTool({
    name: "todo_read",
    description: "Read current tasks from the TodoBoard",
    execute: async () => {
      return await readTasksFromFile('TODO.md');
    }
  });

  pi.registerTool({
    name: "todo_claim",
    description: "Claim a task for this agent",
    parameters: { taskId: "string" },
    execute: async ({ taskId }) => {
      return await claimTask(taskId, pi.agentId);
    }
  });

  pi.registerTool({
    name: "todo_complete",
    description: "Mark a task as complete",
    parameters: { taskId: "string", result: "string" },
    execute: async ({ taskId, result }) => {
      return await completeTask(taskId, pi.agentId, result);
    }
  });

  pi.registerTool({
    name: "todo_add",
    description: "Add a new task to the TodoBoard",
    parameters: { content: "string", priority: "string" },
    execute: async ({ content, priority }) => {
      return await addTask({ content, priority });
    }
  });

  // Register command for human interaction
  pi.registerCommand("tasks", {
    description: "View and manage TodoBoard",
    execute: async () => {
      return await renderTaskBoard();
    }
  });
}
```

2. Supervisor Delegation Protocol

Implement explicit delegation with bounded context:

```
// extensions/supervisor/index.ts
export default function(pi: ExtensionAPI) {
  pi.registerTool({
    name: "delegate_task",
    description: "Delegate a task to a specialized subagent",
    parameters: {
      subagentName: "string",
      task: "object",
      context: "object"
    },
    execute: async ({ subagentName, task, context }) => {
      // Create bounded context for subagent
      const boundedContext = {
        task: task,
        relevantFacts: context.relevantFacts || [],
        constraints: context.constraints || [],
        outputSchema: context.outputSchema
      };

      // Spawn or invoke subagent
      const result = await pi.invokeSubagent(subagentName, boundedContext);

      // Log delegation for audit
      await logDelegation({
        supervisor: pi.agentId,
        subagent: subagentName,
        taskId: task.id,
        timestamp: new Date().toISOString()
      });

      return result;
    }
  });
}
```

3. Shared Memory Extension

Implement namespace-based shared memory with access control:

```
// extensions/shared-memory/index.ts
export default function(pi: ExtensionAPI) {
  const memory = new SharedMemoryStore();

  pi.registerTool({
    name: "memory_write",
    description: "Write to shared memory namespace",
    parameters: { namespace: "string", key: "string", value: "any" },
    execute: async ({ namespace, key, value }) => {
      if (!memory.canWrite(pi.agentId, namespace)) {
        throw new Error(`No write access to ${namespace}`);
      }
      await memory.write(namespace, key, value, pi.agentId);
      return { success: true };
    }
```

```
  });

  pi.registerTool({
    name: "memory_read",
    description: "Read from shared memory namespace",
    parameters: { namespace: "string", key: "string" },
    execute: async ({ namespace, key }) => {
      if (!memory.canRead(pi.agentId, namespace)) {
        throw new Error(`No read access to ${namespace}`);
      }
      return await memory.read(namespace, key);
    }
  });
}
```

8.2 Git Coordination Enhancements

The existing Git rules in AGENTS.md (AGENTS.md) provide strong foundations. Additional enhancements to consider:

File ownership tracking: Maintain a registry of which agent is currently working on which files. Before an agent modifies a file, check that no other agent has claimed it.

Automatic conflict detection: Before committing, run `git diff` against remote to detect potential conflicts. If conflicts are detected, alert the supervisor agent rather than failing silently.

Commit attribution: Include agent ID in commit messages using a consistent format (e.g., `[agent:researcher] feat: add market analysis`). This enables filtering and analysis of agent activity.

Branch-per-task pattern: For complex tasks, each subagent works on a dedicated branch. The supervisor manages merging when tasks complete.

8.3 AGENTS.md Enhancements for Multi-Agent Coordination

Add the following sections to AGENTS.md to formalize multi-agent coordination:

```
## Multi-Agent Coordination

### Agent Roles and Capabilities
Each agent should have a clearly defined role in its system prompt:
- `researcher`: Information gathering and analysis
- `coder`: Code generation and modification
- `reviewer`: Code review and quality assurance
- `planner`: Task decomposition and coordination

### Task Claiming Protocol
Before working on a task:
1. Check TODO.md for available tasks
2. Claim the task using the todo_claim tool
3. Update task status as you progress
4. Mark complete with results when done

### Context Sharing Rules
- Share only relevant context when delegating
- Use memory_write for facts that other agents need
- Never expose sensitive user data between agents
- Document assumptions in task notes
```

```
### Conflict Resolution
When two agents modify the same resource:
1. The agent with the task claim has priority
2. Escalate to supervisor if priority unclear
3. Use consensus for shared decisions
4. Log all conflicts for review
```

8.4 Performance Monitoring Dashboard

Implement monitoring to track multi-agent coordination health:

```
// Monitor key metrics
interface CoordinationMetrics {
  taskCompletionRate: number;     // % tasks completed vs created
  averageTaskLatency: number;     // Time from creation to completion
  delegationSuccessRate: number;  // % delegations that succeed
  conflictRate: number;           // Conflicts per 100 operations
  tokenEfficiency: number;        // Useful output / total tokens
  consensusAgreementRate: number; // % decisions reaching consensus
}

// Log and analyze coordination events
const eventTypes = [
  'TASK_CREATED',
  'TASK_CLAIMED',
  'TASK_COMPLETED',
  'TASK_FAILED',
  'DELEGATION_STARTED',
  'DELEGATION_COMPLETED',
  'CONFLICT_DETECTED',
  'CONSENSUS_REACHED',
  'CONSENSUS_FAILED'
];
```

9 Analysis and Discussion

9.1 Synthesis of Coordination Patterns

The research reveals a consistent set of patterns across successful multi-agent implementations. At the architectural level, the supervisor-subagent model provides the best balance of coordination efficiency and scalability. This pattern maps naturally to human organizational structures, making it intuitive to design and debug.

The critical enabler for effective coordination is **shared, visible state**. Whether implemented as a TodoBoard (tick-md), Git-based file coordination (pi-mono), or a vector database with namespaces (enterprise systems), the principle remains the same: all agents must be able to observe and claim tasks through a single source of truth.

The Hive Mind documentation's Byzantine Fault Tolerance principles apply even when agents are not adversarial. LLM agents can fail unpredictably—hallucinating, timing out, or producing malformed outputs. The requirement that $n \geq 3f + 1$ suggests that critical decisions should involve multiple agents with consensus mechanisms, rather than relying on any single agent.

9.2 Trade-offs and Considerations

Coordination overhead vs. parallelism: Adding coordination mechanisms introduces overhead. The 1.5x-7x token overhead documented in research represents a real cost. Teams must balance the benefits of multi-agent parallelism against coordination costs. For simple, sequential tasks, a single well-designed agent may outperform multiple coordinated agents.

Centralization vs. resilience: Supervisor-subagent architectures create potential single points of failure. If the supervisor fails, coordination breaks down. Mitigation strategies include supervisor redundancy, checkpointing, and graceful degradation to independent agent operation.

Context sharing vs. encapsulation: Sharing more context improves coordination but increases token usage and risks information leakage. Bounded context delegation requires careful thought about what information each agent truly needs.

Determinism vs. flexibility: Deterministic task allocation prevents conflicts but may miss opportunities for dynamic optimization. Some systems benefit from allowing agents to self-select tasks based on current capabilities and context.

9.3 Limitations of This Research

Source recency: While sources range from 2024 to 2026, the multi-agent LLM field evolves rapidly. Specific framework recommendations may become outdated as new versions are released.

Implementation specificity: Recommendations are tailored to the pi-mono harness architecture. Teams using different frameworks will need to adapt patterns to their specific contexts.

Empirical validation: While recommendations are grounded in documented best practices and theoretical foundations, quantitative performance comparisons across coordination strategies were not found in the research. Teams should instrument their implementations to validate that coordination improvements translate to measurable outcome gains.

Scale limitations: Most documented implementations involve relatively small numbers of agents (2-10). Coordination patterns for large-scale systems (100+ agents) may require different approaches not covered in this research.

10 Conclusions

10.1 Key Findings Summary

1. Supervisor-subagent architecture is the dominant pattern: Successful multi-agent implementations consistently use hierarchical coordination with a supervisor managing specialized subagents. This pattern reduces n-to-n communication complexity and provides clear accountability.

2. TodoBoard coordination eliminates task conflicts: Shared, visible task state through TodoBoard-style systems prevents duplicate work, enables deterministic assignment, and provides audit trails for debugging. The tick-md model demonstrates that simple Markdown-based task files can provide effective coordination.

3. Bounded context delegation is essential: Passing complete conversation history to subagents creates token inefficiency and coordination noise. Successful implementations explicitly define what context each subagent receives.

4. Production systems require explicit boundaries: Token limits, time caps, and resource budgets prevent coordination deadlocks. Without these boundaries, multi-agent systems can consume 1.5x-7x more resources than necessary.

5. Git coordination rules work for parallel agents: The pi-mono AGENTS.md Git rules demonstrate that explicit, enforceable coordination rules enable parallel agent work without conflicts. Key rules include agent-specific commits, no blanket staging, and prohibition of destructive Git operations.

10.2 Recommendations

For immediate implementation:
1. Add a TODO.md file to projects with a defined task schema
2. Implement task claiming with file locking to prevent conflicts
3. Extend AGENTS.md with multi-agent coordination rules
4. Add delegation tools to the supervisor agent

For medium-term enhancement:
1. Build a shared memory extension with namespace-based access control
2. Implement event sourcing for coordination audit trails
3. Add monitoring for coordination metrics
4. Create consensus mechanisms for critical decisions

For long-term evolution:
1. Integrate continuous learning from coordination failures
2. Develop adaptive orchestration that optimizes agent selection at runtime
3. Build visualization tools for coordination debugging
4. Implement cross-session persistent memory for collective learning

10.3 Future Research Directions

Several areas warrant additional investigation:

Emergent coordination: Can agents develop their own coordination protocols through interaction, rather than relying on explicitly programmed patterns?

Scale optimization: What coordination patterns are most effective for systems with 50+ agents working on large projects?

Cross-framework interoperability: How can agents built on different frameworks (pi-mono, CrewAI, AutoGen) coordinate effectively?

Human-agent team dynamics: How do coordination patterns change when human team members are mixed with AI agents?

11 References

All sources cited in this report:

1. Galileo AI - 10 Multi-Agent Coordination Strategies - Production coordination strategies and failure prevention
2. AGENTS.md - pi-mono Repository - Git coordination rules for parallel agents
3. packages/coding-agent - pi-mono Repository - Coding agent architecture and subagent extensions
4. Mem0 - CrewAI Guide Multi-Agent AI Teams - CrewAI implementation with persistent memory
5. Olamendy, Medium - Integrated Best Practices - Context compaction and hierarchical memory systems
6. Purple Horizons - tick-md Multi-Agent Coordination - Markdown-based task management for AI agents
7. SparkCo - Deep Dive into Agent Task Decomposition - Task decomposition techniques and best practices
8. SuperAnnotate - Multi-agent LLMs in 2025 - Framework comparisons and collaboration patterns
9. VoltAgent - Multi-Agent LLM Systems in 2025 - Supervisor-subagent architecture implementation
10. DigitalOcean - CrewAI Role-Based Agent Orchestration - Role specialization and task decomposition
11. Hive Mind Multi-Agent Coordination Theory Reference Documentation v1.0 - Theoretical foundations including BFT, CNP, HTN Planning, and Pareto Optimization

12 Appendix A: TodoBoard Schema Reference

Complete schema for TodoBoard task management:

```yaml
# TODO.md YAML frontmatter schema
tasks:
  - id: string              # Unique identifier (TASK-001)
    content: string         # Task description
    status: enum            # pending | in_progress | completed | blocked
    priority: enum          # low | medium | high
    assignee: string?       # Agent ID or name
    blockedBy: string[]?    # Array of blocking task IDs
    createdAt: datetime     # ISO 8601 timestamp
    updatedAt: datetime     # ISO 8601 timestamp
    completedAt: datetime?  # ISO 8601 timestamp
    tags: string[]?         # Categorization tags
    notes: string[]?        # Progress notes with timestamps
```

Example TODO.md:

```yaml
---
tasks:
  - id: TASK-001
    content: "Implement authentication module"
    status: completed
    priority: high
    assignee: coder-agent
    createdAt: "2026-03-05T10:00:00Z"
    updatedAt: "2026-03-05T14:30:00Z"
    completedAt: "2026-03-05T14:30:00Z"
    tags: [backend, security]

  - id: TASK-002
    content: "Write unit tests for authentication"
    status: in_progress
    priority: medium
    assignee: tester-agent
    blockedBy: []
    createdAt: "2026-03-05T14:35:00Z"
    updatedAt: "2026-03-05T15:00:00Z"
    tags: [testing, backend]
    notes:
      - "[2026-03-05T15:00:00Z] Started test implementation"

  - id: TASK-003
    content: "Deploy to staging environment"
    status: blocked
    priority: low
    blockedBy: [TASK-002]
    createdAt: "2026-03-05T10:00:00Z"
    updatedAt: "2026-03-05T10:00:00Z"
    tags: [devops]
---

# Project Tasks

## In Progress
- [~] TASK-002: Write unit tests for authentication (@tester-agent)
```

```
## Blocked
- [ ] TASK-003: Deploy to staging environment (blocked by TASK-002)

## Completed
- [x] TASK-001: Implement authentication module (@coder-agent)
```

13 Appendix B: Coordination Event Types

Standard event types for event-sourced coordination:

```
// Core coordination events
type CoordinationEvent =
  // Task lifecycle
  | { type: 'TASK_CREATED'; taskId: string; content: string;
      priority: string; createdBy: string; timestamp: string }
  | { type: 'TASK_CLAIMED'; taskId: string; agentId: string;
      timestamp: string }
  | { type: 'TASK_PROGRESS'; taskId: string; agentId: string;
      note: string; timestamp: string }
  | { type: 'TASK_COMPLETED'; taskId: string; agentId: string;
      result: any; timestamp: string }
  | { type: 'TASK_FAILED'; taskId: string; agentId: string;
      error: string; timestamp: string }
  | { type: 'TASK_RELEASED'; taskId: string; agentId: string;
      reason: string; timestamp: string }

  // Delegation events
  | { type: 'DELEGATION_INITIATED'; supervisorId: string;
      subagentId: string; taskId: string; context: any; timestamp: string }
  | { type: 'DELEGATION_ACCEPTED'; subagentId: string;
      taskId: string; timestamp: string }
  | { type: 'DELEGATION_COMPLETED'; subagentId: string;
      taskId: string; result: any; timestamp: string }
  | { type: 'DELEGATION_REJECTED'; subagentId: string;
      taskId: string; reason: string; timestamp: string }

  // Memory events
  | { type: 'MEMORY_WRITTEN'; namespace: string; key: string;
      agentId: string; timestamp: string }
  | { type: 'MEMORY_READ'; namespace: string; key: string;
      agentId: string; timestamp: string }

  // Consensus events
  | { type: 'CONSENSUS_INITIATED'; topic: string;
      participants: string[]; timestamp: string }
  | { type: 'CONSENSUS_VOTE'; topic: string; agentId: string;
      vote: any; confidence: number; timestamp: string }
  | { type: 'CONSENSUS_REACHED'; topic: string; decision: any;
      agreement: number; timestamp: string }
  | { type: 'CONSENSUS_FAILED'; topic: string;
      reason: string; timestamp: string }

  // Resource events
  | { type: 'RESOURCE_ACQUIRED'; resource: string;
      agentId: string; timestamp: string }
  | { type: 'RESOURCE_RELEASED'; resource: string;
```

```
    agentId: string; timestamp: string }
  | { type: 'RESOURCE_CONTENTION'; resource: string;
    waitingAgents: string[]; timestamp: string }

  // Checkpoint events
  | { type: 'CHECKPOINT_CREATED'; checkpointId: string;
    state: any; timestamp: string }
  | { type: 'ROLLBACK_INITIATED'; targetCheckpoint: string;
    reason: string; timestamp: string }
  | { type: 'ROLLBACK_COMPLETED'; targetCheckpoint: string;
    timestamp: string };
```

14 Appendix C: Multi-Agent Coordination Checklist

Pre-deployment checklist for multi-agent systems:

Category	Item	Status
Architecture	Supervisor-subagent hierarchy defined	
Architecture	Agent roles clearly documented	
Architecture	Communication topology specified	
Task Management	TodoBoard schema implemented	
Task Management	Task claiming with locking	
Task Management	Dependency tracking enabled	
Memory	Shared memory with namespaces	
Memory	Access control lists defined	
Memory	Context compaction strategy	
Boundaries	Token limits per agent/task	
Boundaries	Time caps for operations	
Boundaries	Idle timeout handling	
Guardrails	Runtime policy enforcement	
Guardrails	Action verification for high-impact ops	
Guardrails	Audit logging enabled	
Recovery	Checkpointing at strategic points	
Recovery	Rollback procedures documented	
Recovery	Conflict resolution protocol	
Monitoring	Coordination metrics tracked	
Monitoring	Failure pattern detection	
Monitoring	Dashboard for visibility	